[Short Paper] One Query per Template: Moving Query Runtime Models to New Hardware

Krish Patel
kmp.patel@uwaterloo.ca
University of Waterloo
Waterloo, Canada

Abstract

A model that predicts query runtimes is trained on measurements from one machine. When the database moves to other hardware, its plans stay the same but their runtimes change. We ask how to adapt such a model from k queries measured on the new machine. On PostgreSQL we measured 1,162 TPC-H and TPC-DS queries in six environments that run identical plans: a laptop and a server, each with all processors and with PostgreSQL pinned to one processor, the server also pinned to two, and the laptop with a small buffer pool. The slowdown between two environments is far from a single factor: most of its variance lies between query templates, and for changes of processor count or hardware, whether a plan runs in parallel and how long it runs explain much of it. A single speed factor therefore stops improving after a few queries. Our correction recalibrates serial and parallel plans with separate lines and adds an offset for every measured template. With one measured query per template it brings the moved model's median Q-error, averaged over templates, to 1.10 on TPC-H and 1.19 on TPC-DS, close to what it gives a model trained on the new machine (1.08 and 1.18): once every template has a query, the move itself costs little, and the separate lines matter mainly before that point, more on TPC-H than on TPC-DS. On templates never measured, no correction does much better than the speed factor, whose error is close to that of retraining. We fixed five hypotheses before the labels of 18 pairs existed; all five hold as registered, though one falls short when counted by confidence intervals.

Keywords

query performance prediction, learned cost models, hardware change, model adaptation, PostgreSQL

1 Introduction

Predicting how long a query will run before it runs serves admission control, scheduling, provisioning and the choice between plans. Learned models predict runtimes from plan features far better than raw optimizer costs when they are trained on the system they predict [1, 6, 20, 29]. A runtime, however, is a property of a plan on a machine, and machines change: a database moves to new hardware, to a cloud instance of another size, or into a container with fewer processors. The plans stay, and every runtime the model learned from is now off, by an amount that depends on the query.

Systems answer this in three ways. Some retrain on the new machine: T3 reruns its whole benchmark [25], and QCFE reruns simplified queries to refit its per-operator coefficients and then retrains on 200 labeled queries [39]. Some map every old prediction through one fitted line, the model shift used to move performance models of configurable software [13, 33], whose simplest form rescales every prediction by one speed factor. Others describe the machine with features and train one model over many machines, as Redshift's Stage predictor does with the instance type [37]. This paper asks what lies between retraining and rescaling: if we can afford to run k queries on the new machine, which queries should they be, how should their runtimes be used, and how large must k be?

We study the question with the plans held fixed. We measured 1,162 queries from 115 templates of TPC-H and TPC-DS on PostgreSQL in six environments that run identical plans on copies of the same database. Their 30 ordered pairs, each measured on both workloads, change the processor count, the hardware, both, the buffer pool, or several of these at once. We contribute:

- A measurement of how runtimes shift when only the environment changes (Section 4.1). The slowdown differs widely between templates. Whether a plan has parallel workers carries much of a change of processor count, and how long the query runs carries much of the move from laptop to server; per-operator timings point to the mechanisms.
- A comparison of corrections, baselines and query selections for 5 to 200 measured queries (Sections 4.2 and 4.3). One measured query per template with a per-template offset brings the moved model close to the same correction on a model trained on the new machine; refitting the model on the measured queries is far worse at these budgets.
- A negative result for new templates (Section 4.4): no correction does much better than the speed factor, whose error is close to that of retraining, so the remaining error is the model's error on a new template, and the move adds little to it.
- A confirmatory test (Section 4.6): we committed five hypotheses and the primary method to our repository before any label of 18 of the pairs existed, and all five hold under the rules we fixed.

2 Problem and Methods

Setting. A model f_A predicts the log runtime of a query from features x of its PostgreSQL plan before the query runs, and was trained on the runtimes of a pool of queries measured on machine A. The database moves to machine B with the same data, statistics and planner settings, so the planner chooses the same plans; we call y_B/y_A a query's slowdown. Before f_A is used on B we may run k queries there. Every query belongs to a template, the parameterized SQL it was generated from; in a production system templates can be read from query fingerprints such as PostgreSQL's query identifier.

Features and source model. The 19 plan features are the root's startup and total cost, estimated rows and width; the number of nodes and the depth; the largest row estimate of any

EDBT '27, Lille (France)
© 2027 Copyright held by the owner/author(s). Published on xxxxx under ISBN xxx-x-xxxxx-xxx-x, series ISSN xxxx-xxxx. Distribution of this paper is permitted under the terms of the Creative Commons license CC-by-nc-nd 4.0.

node and the sum of the nodes' log row estimates; the counts of all scans, sequential scans, index scans (bitmap scans included), nested-loop, hash and merge joins, sorts, aggregates and materializations; the workers planned over all Gather nodes; and the number of subplans. Costs, rows and widths enter as $\log(1+x)$. The source model fits a line in $\log(1+\text{total cost})$ to the log runtime (the cost line) and a random forest to its residuals (300 trees, at least two queries per leaf, 4 of the 19 features tried at each split) [2, 24]. Section 4.5 reports four other source models.

Corrections. All corrections act on the log errors of the k measured queries, $e_i = \log y_i - f_A(x_i)$, and predict $\log \hat{y} = f_A(x) + c(x)$ on B; fitting a learner to the residuals of a base model follows [3].

- *Speed factor:* c is the mean of the e_i.
- *Affine+parallel:* $c(x) = a + b\, f_A(x) + \pi(x)\,(a' + b'\, f_A(x))$, where $\pi(x)$ is 1 for plans with parallel workers; b makes the slowdown depend on the predicted runtime. The intercept a' enters only when both kinds of plan are among the k queries, the slope b' only when each kind has two. The fit is ridge regression [12] with an unpenalized intercept, its penalty chosen by leave-one-out [7] from 45 values between 10^{-6} and 10^{5}, so a large penalty gives back the speed factor.
- *Ridge correction:* ridge regression of e on the 19 standardized features, fitted the same way.
- *Template offsets:* the speed factor plus, for every template among the k queries, the mean residual of its queries from that factor; other templates get the factor alone.
- *Low-dim + template*, our primary method: affine+parallel plus, for every template among the k queries, the mean residual of its queries from that line; other templates get the line alone.
- *1-NN:* the error of the nearest of the k queries in the standardized feature space. It needs no template labels.

We also compare against two refits, a nonlinear correction and a lookup: *target only* refits the source pipeline on the k queries alone; *pooled* refits it on the source pool and the k queries with the machine as a feature; *forest on the errors* fits a forest to the e_i; and a *template lookup* predicts a query by the mean measured log runtime of its template, with no model. The *oracle* refits the source pipeline on the whole target pool. It is a reference, not a bound: it knows nothing of templates. The *no-move control* applies a method to a model trained on the target pool itself, with the same k queries; the gap to it is the cost of the move.

Choosing the k queries. *Random* takes them in a random order. *Round robin* takes one query of each template in turn, in a random order of templates and of queries; *cheapest per template* is round robin with the query that ran fastest on A first. *k-center* adds the query farthest from those chosen so far in the standardized feature space [8, 26]. *D-optimal* adds the query that most increases $\det(I + X^\top X)$, where X holds the chosen queries' standardized features and a column of ones [22, 38]. *Plan class* alternates between parallel and serial plans. Every strategy derives its r-th sequence from the r-th of 20 random orders, so comparisons are paired; none looks at a runtime on B.

3 Experimental Setup

Environments. M1 is a laptop (Intel Core Ultra 7 155H, 22 threads) running PostgreSQL 18.6 in WSL2 (a Linux virtual machine on Windows) with 16 GB of memory; M3 is a virtual machine with 64 virtual processors of an AMD EPYC 9754 and 1 TB of memory on a shared university server. M2 and M4 are M1 and M3 with every PostgreSQL server process pinned to one virtual processor, so that a plan's leader and its parallel workers (at most two per Gather node) share it while the client keeps all processors; M6 is M3 pinned to two. M5 is M1 with PostgreSQL's default buffer pool, 128 MB instead of 2 GB. All use the same PostgreSQL build, which has no JIT, and the same settings (work_mem 64 MB, effective_cache_size 8 GB) apart from M5's buffer pool, on copies of the same database files; every saved plan, with its estimates and costs, is identical in all six. The data fit in memory everywhere.

Workloads. TPC-H at scale factor 1 [30]: 680 queries from 22 templates, 420 from its query generator and 260 with range parameters drawn wider than the specification's, so that runtimes vary inside a template. TPC-DS at scale factor 1 [31]: 482 queries from the 93 templates whose first query ran within 30 s on M1; in its four templates of two statements, each statement counts as a query of the template. Queries are split once, inside every template, into a pool from which the k queries are drawn (476 and 294 queries, 70% and 61%) and a test set on which the corrections are scored. We link two TPC-DS templates when their first queries, with literals masked, share at least 85% of their tokens, and take the connected components as families: 75 families, 15 of them with more than one template.

Labels. Every environment runs every query in three passes in different random orders; in a pass a query runs four times with `EXPLAIN (ANALYZE, TIMING OFF)`, the first only to warm the cache. A run's time is the execution time PostgreSQL reports, without planning or sending rows to the client, and a label is the median over passes of the median of the three timed runs. A pass whose level shifts by more than 10% partway through, or during which the laptop slept or ran on battery, is measured once more. The median Q-error between two passes of the same query is 1.03 to 1.09 per environment and workload. For Section 4.1 we also ran every query once more in M1 to M5 with per-node timing on, after a warm-up; an operator's own time is its time over all loops minus its children's, and these timings are never labels.

Measures. The error of a test template is the median Q-error ($\max(\hat{y}/y, y/\hat{y})$) of its test queries; a method's error is the geometric mean of the template errors, averaged in log over 20 random orders of the pool (5 in the test on new templates). Ratios between two methods are geometric means over templates of the ratio of their errors, with 95% intervals from 2000 resamples of templates (of families in TPC-DS), paired across methods; one method *beats* another when that interval lies below 1. The test on new templates holds out one template at a time (in TPC-DS its family): the source model is retrained on A's pool without it, the $k = 25$ queries are drawn from B's pool without it, the oracle is retrained on that pool, and the template's test queries are scored.

Exploratory and confirmatory analyses. We designed the corrections and strategies on TPC-H with M1, M2 and M3, so those pairs are exploratory. We committed the primary method (low-dim + template with round-robin selection) and five hypotheses (commit 17c328f) after the first TPC-DS passes on M2 and M3 had started and before any label of TPC-DS on M2, M3 or M4 or of TPC-H on M4 existed; Section 4.6 tests them on the 18 pairs these labels form. Within twenty minutes, still before any such label existed, a code review led to an amendment (commits b2e59dc and fc9dc9a): TPC-DS intervals resample families, the test on new templates holds out families, and affine+parallel and the primary method leave out their parallel terms when the k queries hold one plan class. The per-operator timings and the

Table 1: How the runtime of the same plan changes, for seven of the 30 ordered pairs of each workload. Slowdown: median of target over source runtime, for serial plans and plans with parallel workers. Template share: the share of the variance of the log slowdown between templates (shift), the same corrected for the number of templates (ϵ^2), and the share between two passes on the source alone (noise). R^2: least-squares fits of the log slowdown on whether the plan is parallel, on the log source runtime (size) and on both with their interaction; features (new): a ridge model on the 19 features scored on templates (TPC-DS: families) left out of its fit, which can be negative.

		Slowdown		Template share			R^2 of the log slowdown on			
Pair	Change	serial	parallel	shift	ϵ^2	noise	parallel	size	both	features (new)
TPC-H										
M1→M2	processors	1.12	2.66	0.86	0.86	0.08	0.61	0.00	0.61	0.27
M3→M4	processors	1.01	2.07	0.84	0.83	0.03	0.57	0.12	0.62	0.53
M3→M6	two processors	1.00	1.32	0.69	0.68	0.03	0.51	0.06	0.55	0.31
M1→M3	hardware	1.81	2.07	0.80	0.79	0.08	0.07	0.52	0.63	0.45
M2→M4	hardware	1.63	1.69	0.74	0.73	0.03	0.02	0.22	0.33	0.41
M1→M4	both	1.82	4.42	0.91	0.90	0.08	0.54	0.03	0.60	0.26
M1→M5	buffer pool	1.47	1.07	0.72	0.71	0.08	0.03	0.03	0.08	-0.12
TPC-DS										
M1→M2	processors	1.06	2.11	0.91	0.89	0.18	0.44	0.00	0.50	0.26
M3→M4	processors	0.92	2.05	0.94	0.93	0.18	0.56	0.10	0.59	0.38
M3→M6	two processors	0.89	1.25	0.88	0.85	0.18	0.33	0.19	0.44	0.30
M1→M3	hardware	2.06	1.80	0.83	0.80	0.18	0.07	0.64	0.65	0.20
M2→M4	hardware	1.80	1.77	0.85	0.81	0.22	0.01	0.41	0.45	-0.05
M1→M4	both	1.83	3.79	0.92	0.90	0.18	0.34	0.06	0.52	0.19
M1→M5	buffer pool	0.98	1.03	0.77	0.72	0.18	0.00	0.02	0.02	-0.06

results of Section 4.5 are exploratory, as are M5, M6, the no-move control, the lookup and the single-run labels of Section 4.3, which were added later.

4 Results

4.1 How runtimes shift

Table 1 shows seven pairs per workload. No single factor describes any of them. Over all pairs, a share of 0.69 to 0.94 of the variance of the log slowdown lies between templates (0.68 to 0.93 after correcting for the number of templates), against at most 0.08 (TPC-H) and 0.22 (TPC-DS) between two passes of one machine. What separates the templates depends on the change. Pinning PostgreSQL to one processor roughly doubles the runtime of plans with parallel workers and leaves serial plans almost unchanged, and that one bit explains about half of the variance; with two processors (M3→M6) parallel plans slow by only 1.32 (TPC-H) and 1.25 (TPC-DS). Moving from the laptop to the server slows serial and parallel plans alike, but short queries more than long ones: the log slowdown falls as the log runtime grows, and size explains 0.52 (TPC-H) and 0.64 (TPC-DS) of it. The small buffer pool moves the median runtime by a factor of only 1.08 (TPC-H) and 1.03 (TPC-DS), although TPC-H's serial plans slow by 1.47, and no coarse plan property explains it.

Per-operator timings point to the mechanisms (Table 2), although per-node timing adds overhead that differs between machines. On one processor, the main operators of each workload (at least 3% of its time on the source; Table 2 pools the workloads and also lists smaller ones) take 2.0 to 3.2 times as long inside the parallel part of a plan on the laptop and 2.0 to 3.0 times on the server, while serial operators keep their speed: the same mechanism on both machines. The move from laptop to server slows operators unevenly, by 1.0 to 2.0 times depending on the operator, so plans with different operators slow differently. With the small buffer pool, index scans, which revisit pages that no longer stay in PostgreSQL's buffers, slow down 1.3 to 1.9 times, while sequential scans, which read pages in order, change by 1.0 to 1.3. Which templates slow down thus depends on their operators, which plan-level features see only in part: a ridge model on the 19 features, fitted without the template it predicts, explains at most 0.53 of the variance, and nothing for the buffer pool (last column of Table 1).

Table 2: Median ratio of each operator's own time (target over source), inside (par.) and outside (ser.) the parallel part of plans, over the nodes of both workloads weighted by the source's time, from one extra pass with per-node timing. Proc.: one processor (M1→M2, M3→M4); hardware: M1→M3; buffer pool: M1→M5.

	Proc. (laptop)		Proc. (server)		Hardware		Buffer pool	
Operator	par.	ser.	par.	ser.	par.	ser.	par.	ser.
Seq Scan	2.85	0.99	2.85	1.01	1.75	1.70	1.12	1.23
Index Scan	2.33	1.00	2.32	1.07	1.64	1.01	1.92	1.42
Bitmap Heap Scan	1.98	0.99	2.04	1.20	2.01	1.49	0.98	2.14
Hash Join	2.56	1.01	2.57	0.94	1.62	1.65	1.00	1.00
Hash	2.89	1.05	2.32	1.02	1.32	1.34	1.13	1.06
Nested Loop	1.50	1.01	0.93	0.91	1.56	1.72	1.09	1.03
Sort	2.53	1.01	2.59	1.02	1.25	1.43	1.04	1.07
Hash aggregate	2.94	1.12	3.05	1.15	1.43	1.50	0.98	1.08
Sorted aggregate	1.78	1.00	1.99	1.15	1.42	1.58	1.02	1.04

4.2 Adapting with k queries

Figure 1 and Table 3 give the errors, and Table 4 compares the methods at one query per template. Without adaptation the source model errs by 1.83 on TPC-H and 1.96 on TPC-DS. The speed factor finds the average slowdown from a few queries and then stays flat (TPC-H: 1.39 at $k = 5$, 1.37 at $k = 25$; TPC-DS: 1.50 and 1.45), because no single factor fits templates whose slowdowns differ. Affine+parallel helps at small budgets on TPC-H: it beats the speed factor at $k = 10$ in 20 of 30 pairs and loses in 0; for processor changes it lowers the $k = 10$ error from 1.41 to

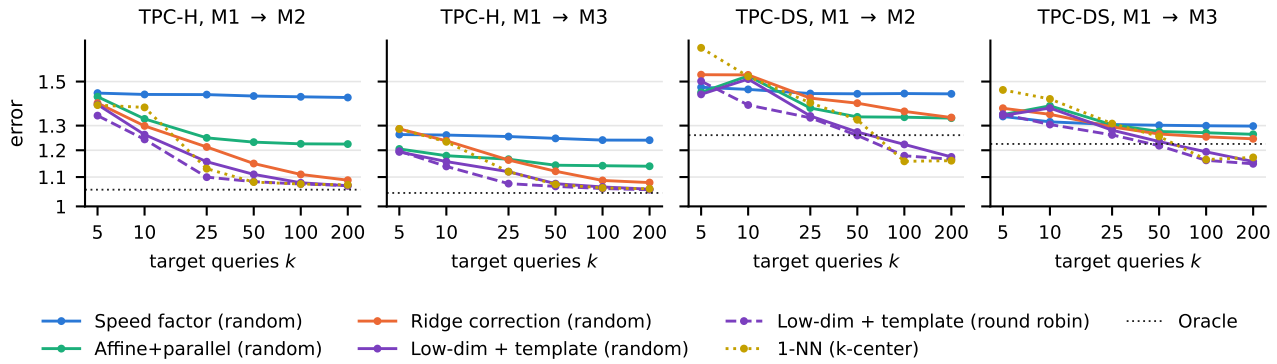


Figure 1: Error (geometric mean over test templates of the median Q-error) against the number k of measured target queries, both axes logarithmic, for a processor change (M1→M2) and a hardware change (M1→M3) on each workload; the TPC-H panels are pairs the methods were designed on. Line style gives the selection: solid random, dashed round robin, dotted k-center. The horizontal dotted line is the oracle.

Table 3: Error by kind of change, as the geometric mean over its ordered pairs. Speed, Aff.+par. and Ridge use random selection; Primary (low-dim + template) uses random selection at $k = 10$ and round robin at $k = 25$ and $k = 100$. Oracle: the source pipeline refitted on the whole target pool. New templates: each template (TPC-DS: its family) held out from the source model, the $k = 25$ randomly chosen queries, the oracle and every correction. Mixed: the remaining 14 pairs, all involving M5 or M6.

		$k = 10$, random			$k = 25$		$k = 100$		New templates, $k = 25$			
Change	Pairs	Speed	Aff.+par.	Primary	Ridge	Primary	Primary	Oracle	Speed	Aff.+par.	Ridge	Oracle
TPC-H												
processors	4	1.41	1.28	1.23	1.20	1.09	1.07	1.05	1.99	2.00	2.13	1.98
two processors	2	1.14	1.12	1.11	1.12	1.08	1.06	1.05	1.76	1.76	1.77	1.76
hardware	4	1.22	1.19	1.17	1.15	1.09	1.07	1.05	1.99	1.98	1.99	1.98
both	4	1.46	1.33	1.26	1.21	1.10	1.08	1.05	2.01	2.01	2.11	1.98
buffer pool	2	1.38	1.39	1.32	1.33	1.13	1.10	1.05	2.16	2.14	2.47	2.11
mixed	14	1.44	1.35	1.29	1.29	1.11	1.08	1.05	2.08	2.03	2.27	2.03
TPC-DS												
processors	4	1.52	1.49	1.47	1.49	1.35	1.18	1.25	2.72	2.65	2.73	2.63
two processors	2	1.36	1.34	1.33	1.33	1.28	1.16	1.23	2.55	2.54	2.59	2.52
hardware	4	1.34	1.35	1.35	1.34	1.28	1.18	1.25	2.64	2.64	2.67	2.63
both	4	1.55	1.51	1.49	1.55	1.37	1.18	1.25	2.72	2.65	2.74	2.63
buffer pool	2	1.47	1.49	1.48	1.46	1.41	1.24	1.31	3.01	3.01	3.05	2.99
mixed	14	1.50	1.49	1.47	1.50	1.38	1.19	1.27	2.78	2.75	2.85	2.73

1.28, and the primary method's template offsets bring it to 1.23. On TPC-DS, affine+parallel beats the speed factor at $k = 10$ in 10 pairs and loses in 7, and its lead grows with k (Figure 1).

Once each template has a measured query, the template offsets carry the gain. With round-robin selection the primary method reaches 1.10 on TPC-H at $k = 25$ and 1.19 on TPC-DS at $k = 100$; offsets on the speed factor alone give 1.12 and 1.19, so the affine+parallel line lowers the error by about 1% on TPC-H and by almost nothing on TPC-DS. The same correction on a model trained on the target itself reaches 1.08 and 1.18: the move costs little. Without the correction, that model is the oracle, which refits the template-blind pipeline on the whole target pool. On TPC-H the oracle errs by only 1.05, so there the correction itself costs about 3%; on TPC-DS it reaches only 1.26, since three pool queries per template are too few for its forest, and the primary method is 0.93 to 0.95 times the oracle's error in every pair. The model matters inside a template: a lookup of the measured runtime per template errs by 1.24 on TPC-H against 1.10 for the primary method, because the widened queries vary inside their templates; on TPC-DS, whose queries vary inside a template by a factor of only 1.09 (median geometric standard deviation), the gap is smaller (1.24 against 1.19). The refits and the forest on the errors ran with random selection only. With the same selection the primary method reaches 1.17 and 1.23 (Table 5), against 1.51 and 1.80 for target only, 1.36 and 1.36 for the pooled refit, and 1.22 and 1.28 for a forest on the errors.

4.3 Which queries to measure

Table 5 compares the strategies. Covering the templates is what matters. On TPC-H at $k = 25$, round robin lowers the primary method's error from 1.17 to 1.10. Without template labels, k-center selection with the 1-NN correction comes close on TPC-H (1.13) and matches the primary method on TPC-DS at $k = 100$ (1.18 against 1.19, lower in 22 pairs), since queries of one template have near plans. D-optimal selection, which is built for the ridge correction, gives that correction its lowest TPC-DS error, but on

Table 4: Error at the budget where each template can have one query, as the geometric mean over the 30 ordered pairs of a workload; the last row applies the primary method to a model trained on the target itself (no move).

Method	Selection	TPC-H $k = 25$	TPC-DS $k = 100$
No adaptation		1.83	1.96
Speed factor	random	1.37	1.45
Ridge correction	random	1.24	1.38
Target only	random	1.51	1.80
Pooled refit	random	1.36	1.36
Forest on the errors	random	1.22	1.28
Template lookup (no model)	round robin	1.24	1.24
Template offsets	round robin	1.12	1.19
1-NN	round robin	1.10	1.19
Primary	round robin	1.10	1.19
Oracle (whole target pool)		1.05	1.26
No move: primary	round robin	1.08	1.18

Table 5: Error of three corrections (columns) when the k queries are chosen by each strategy (rows), as the geometric mean over the 30 ordered pairs of a workload, at the smallest budget of our grid that holds one query per template. Seconds: total runtime of the k queries on the target, one run each.

Strategy	Primary	1-NN	Ridge	Seconds
TPC-H, $k = 25$				
random	1.17	1.19	1.24	22
round robin	1.10	1.10	1.17	24
cheapest per template	1.17	1.17	1.23	16
k-center	1.13	1.13	1.21	21
D-optimal	1.14	1.13	1.20	16
plan class	1.19	1.21	1.26	18
TPC-DS, $k = 100$				
random	1.23	1.29	1.38	127
round robin	1.19	1.19	1.37	110
cheapest per template	1.22	1.23	1.38	88
k-center	1.18	1.18	1.36	132
D-optimal	1.28	1.39	1.35	188
plan class	1.24	1.30	1.41	118

TPC-DS it picks long queries (187.7 s against 109.9 s for round robin) and worsens the primary method; on TPC-H it picks the cheapest set. Measuring the cheapest query of each template saves about a quarter of the time but costs accuracy, since short queries shift differently from long ones. Running the round-robin queries once takes about 23.6 s on TPC-H and 109.9 s on TPC-DS, while the corrections above use labels from three passes. With one run per query instead, all from one pass as in a single session, the primary method errs by 1.12 on TPC-H and 1.22 on TPC-DS when that run is warm, and by 1.14 and 1.25 when it is the query's first run in the pass; the no-move control rises about as much (to 1.12 and 1.24), so the move still costs little.

4.4 New templates

The last four columns of Table 3 test templates absent from the k queries and from every model's training. Here no correction does much better than the speed factor: affine+parallel is within a few percent of it, and the ridge correction is worse, since its feature slopes, fitted to the measured templates, do not carry to a new one. Retraining helps little: the oracle, retrained on the target pool outside the held-out template, errs by 1.76 to 2.11 on TPC-H and 2.52 to 2.99 on TPC-DS, about as much as the speed factor, whose error is at most 1.12 times the oracle's in any pair. These errors are the model's on a template it has never seen, as plan-level models have long shown [1]; the move to a new machine barely adds to them.

Table 6: The five hypotheses, fixed before any label of the 18 confirmatory pairs existed (the 12 ordered TPC-DS pairs among M1 to M4 and the 6 TPC-H pairs with M4), and their results. H2 counts pairs whose 95% interval lies below 1; H3 and H5 count pairs whose ratio is below 1, as registered; H4 bounds the speed factor's error over the oracle's.

	Hypothesis and result	
H1	Template share above 0.5 in every pair (both directions of a pair share one value); smallest 0.74.	holds
H2	$k = 100$: primary (round robin) beats the speed factor (random) in every pair and the ridge correction (random) in at least two thirds; 18 and 16 of 18.	holds
H3	$k = 10$: affine+parallel (random) has a lower error than the speed factor (random) in at least three quarters of the pairs that change the processor count; 10 of 12.	holds
H4	New templates: the speed factor's error is at most 1.15 times the oracle's in every pair; largest 1.08.	holds
H5	$k = 100$: k-center with 1-NN has a lower error than random selection with the ridge correction in at least two thirds; 18 of 18.	holds

4.5 Other source models and streams

The primary method's advantage does not depend on the source model. With a random forest, gradient-boosted trees, ridge regression or the cost line as f_A, it beats the speed factor at one query per template in all 240 combinations of source model, workload and pair. It also repairs weak models: on the cost line alone, whose oracle errs by 1.87 on TPC-H and 2.61 on TPC-DS, it reaches 1.15 and 1.19. We also replay each pair's test queries as a stream that runs on the source machine and then, without warning, on the target, in batches of about ten queries, each predicted before it is learned from [5]. Over the last five batches, the primary method refitted on the last 50 queries reaches a batch median Q-error of 1.15, against 1.21 for the ridge correction on the same window, 1.49 for the ridge correction refitted on all queries seen, whose history keeps the old machine's errors, and 1.91 without updates.

4.6 Confirmatory tests

Table 6 lists the hypotheses with their results on the 18 pairs whose labels did not exist when we fixed them. All five hold under the registered rules. Counted by intervals instead, H5 still holds (16 of 18 pairs), but H3 has 8 of 12, short of its threshold: the small-budget gain of affine+parallel is real on TPC-H and uncertain on TPC-DS.

5 Related Work

Learned runtime models and hardware. Learned models predict query runtimes from plan features [1, 6, 20, 29, 41], and most are trained for one deployment. T3 moves to new hardware by rerunning its benchmark and retraining [25]; zero-shot cost models transfer across databases on one machine and leave hardware features to future work [11]; a recent drift benchmark and an experimental study cover data and workload drift, not hardware [21, 42]; and a recent tutorial lists aligning label collection with the production hardware as open [10]. Redshift's Stage predictor feeds the instance type to a global model trained across its fleet and backs it with a model and a cache of repeated queries for each instance [37]; Costream takes hardware features as input for stream processing [9]; and Packing Light predicts the throughput of TPC-H and TPC-DS mixes on new cloud hardware from fingerprints taken on a few sampled configurations [4]. The closest work moves a cost model to a second machine: QCFE refits per-operator coefficients from queries run there and retrains on 200 labeled queries [39], DACE fine-tunes on a whole workload rerun on the second machine [17], and ParamTree predicts PostgreSQL's cost constants from a machine's hardware and settings and, on unseen cloud servers, beats both a linearly scaled cost model and one calibrated on samples from each server [40]. Older work calibrates cost formulas from measured queries: Wu et al. calibrate PostgreSQL's cost units on each machine with designed queries [36], and Zhu and Larson fit a regression cost formula per query class of a multidatabase system from queries sampled in that class [43], a precursor of our per-template offsets, as is Cleo's combination of many small cost models for recurring subgraphs with general ones [27]. We add a controlled measurement of how much of a hardware shift a handful of queries can absorb, and of which queries to run.

Transferring performance models. For configurable software, a fitted line from source to target performance (model shift) carries a model across hardware when the change is mild [13, 33], and fails for multithreaded programs or severe changes [13, 14, 32]; Libra, which moves database configuration models across hardware and workloads, finds that model shift can transfer negatively [23]. OneProxy adapts a neural-network latency predictor to a new device from about 50 measurements with one global factor plus sparse per-operator deviations and aims to preserve rankings [18]; our ridge correction has the same form with an L2 penalty. The affine part of our affine+parallel correction is model shift on the log prediction, and our results show where it stops: a change of processor count acts through parallel plans, and every change acts through templates.

Model updating and choosing samples. Clinical prediction research calls the speed factor *recalibration in the large* and finds that such simple updates beat revising the whole model unless the new sample is large [28, 35]. Our ridge correction is regularized least-squares transfer on the source model's residuals, with an unpenalized intercept and a leave-one-out penalty [15]. To choose queries we use greedy D-optimal design [22, 38] and farthest-first selection [8, 26]. Ernest picks the training runs of its linear runtime model by A-optimal design under a budget on their cost [34], which would counter the long queries that D-optimal selection picks here; L2S uses the source model to choose which configurations to measure [14], Warper which queries to use when it adapts a cardinality estimator [16], and Ma et al. which queries to label for learned database components [19].

6 Conclusion

In our study, moving a runtime model to new hardware did not need a new training set. One measured query per template, with an offset for each template on the moved model, came close to the same correction on a model trained on the new machine; separate lines for serial and parallel plans mattered mostly before every template had a query, and more on TPC-H than on TPC-DS. For templates never measured, the speed factor was about as good as anything we tried, because the remaining error is the model's error on a template it has not seen, and the change of machine adds little to it. Our study has limits. It covers one database system on two x86 machines, both virtualized, at scale factor 1, where the data fit in memory in every environment, and only queries under 30 s, run warm and one at a time. We score per-template medians and do not measure tail errors. The 30 pairs of a workload reuse the labels of six environments, so they are not independent replications. The processor changes cut a plan's leader and workers to one or two processors, and pinning keeps the planner's two workers per Gather; a target with more processors than a plan's processes would hide most of that effect. Plans are fixed, so a move that also changes settings or plans is out of scope, and all source models use the same 19 plan-level features. Operator-level models, other machines and data larger than memory are the next steps.

Acknowledgments

Claude (Anthropic) was used to generate sections of this work, including text, tables, graphs, code, data and citations.

Artifacts

The query generators, the measurement and evaluation code, all labels, plans and per-operator timings, the study plan with its timestamped amendments, and the scripts that build every number, table and figure of this paper are at https://github.com/KrishPatel13/query-runtime-transfer. Its README lists the commands: `src/paper_eval.py` runs the evaluation from the labels, `src/paper_report.py` and its companions write the tables, figures and LaTeX macros, and `scripts/` repeats the measurements.

References

[1] Mert Akdere, Ugur Çetintemel, Matteo Riondato, Eli Upfal, and Stanley B. Zdonik. 2012. Learning-based Query Performance Modeling and Prediction. In *Proceedings of the 28th IEEE International Conference on Data Engineering (ICDE)*. 390–401. doi:10.1109/ICDE.2012.64

[2] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324

[3] Diego Didona, Francesco Quaglia, Paolo Romano, and Ennio Torre. 2015. Enhancing Performance Prediction Robustness by Combining Analytical Modeling and Machine Learning. In *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering (ICPE)*. 145–156. doi:10.1145/2668930.2688047

[4] Jennie Duggan, Yun Chi, Hakan Hacıgümüş, Shenghuo Zhu, and Ugur Çetintemel. 2013. Packing Light: Portable Workload Performance Prediction for the Cloud. In *Proceedings of the 29th IEEE International Conference on Data Engineering Workshops (ICDEW)*. 258–265. doi:10.1109/ICDEW.2013.6547460

[5] João Gama, Raquel Sebastião, and Pedro Pereira Rodrigues. 2013. On Evaluating Stream Learning Algorithms. *Machine Learning* 90, 3 (2013), 317–346. doi:10.1007/s10994-012-5320-9

[6] Archana Ganapathi, Harumi A. Kuno, Umeshwar Dayal, Janet L. Wiener, Armando Fox, Michael I. Jordan, and David A. Patterson. 2009. Predicting Multiple Metrics for Queries: Better Decisions Enabled by Machine Learning. In *Proceedings of the 25th IEEE International Conference on Data Engineering (ICDE)*. 592–603. doi:10.1109/ICDE.2009.130

[7] Gene H. Golub, Michael Heath, and Grace Wahba. 1979. Generalized Cross-Validation as a Method for Choosing a Good Ridge Parameter. *Technometrics* 21, 2 (1979), 215–223. doi:10.1080/00401706.1979.10489751

[8] Teofilo F. Gonzalez. 1985. Clustering to Minimize the Maximum Intercluster Distance. *Theoretical Computer Science* 38 (1985), 293–306. doi:10.1016/0304-3975(85)90224-5

[9] Roman Heinrich, Carsten Binnig, Harald Kornmayer, and Manisha Luthra. 2024. Costream: Learned Cost Models for Operator Placement in Edge-Cloud Environments. In *Proceedings of the 40th IEEE International Conference on Data Engineering (ICDE)*. 96–109. doi:10.1109/ICDE60146.2024.00015

[10] Roman Heinrich, Xiao Li, Manisha Luthra, and Zoi Kaoudi. 2025. Learned Cost Models for Query Optimization: From Batch to Streaming Systems. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5482–5487. doi:10.14778/3750601.3750699

[11] Benjamin Hilprecht and Carsten Binnig. 2022. Zero-Shot Cost Models for Out-of-the-box Learned Cost Prediction. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2361–2374. doi:10.14778/3551793.3551799

[12] Arthur E. Hoerl and Robert W. Kennard. 1970. Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics* 12, 1 (1970), 55–67. doi:10.1080/00401706.1970.10488634

[13] Pooyan Jamshidi, Norbert Siegmund, Miguel Velez, Christian Kästner, Akshay Patel, and Yuvraj Agarwal. 2017. Transfer Learning for Performance Modeling of Configurable Systems: An Exploratory Analysis. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 497–508. doi:10.1109/ASE.2017.8115661

[14] Pooyan Jamshidi, Miguel Velez, Christian Kästner, and Norbert Siegmund. 2018. Learning to Sample: Exploiting Similarities Across Environments to Learn Performance Models for Configurable Systems. In *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 71–82. doi:10.1145/3236024.3236074

[15] Ilja Kuzborskij and Francesco Orabona. 2013. Stability and Hypothesis Transfer Learning. In *Proceedings of the 30th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 28)*. 942–950.

[16] Beibin Li, Yao Lu, and Srikanth Kandula. 2022. Warper: Efficiently Adapting Learned Cardinality Estimators to Data and Workload Drifts. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*. 1920–1933. doi:10.1145/3514221.3526179

[17] Zibo Liang, Xu Chen, Yuyang Xia, Runfan Ye, Haitian Chen, Jiandong Xie, and Kai Zheng. 2024. DACE: A Database-Agnostic Cost Estimator. In *Proceedings of the 40th IEEE International Conference on Data Engineering (ICDE)*. 4925–4937. doi:10.1109/ICDE60146.2024.00374

[18] Bingqian Lu, Jianyi Yang, Weiwen Jiang, Yiyu Shi, and Shaolei Ren. 2021. One Proxy Device Is Enough for Hardware-Aware Neural Architecture Search. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 5, 3, Article 34 (2021), 34 pages. doi:10.1145/3491046

[19] Lin Ma, Bailu Ding, Sudipto Das, and Adith Swaminathan. 2020. Active Learning for ML Enhanced Database Systems. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 175–191. doi:10.1145/3318464.3389768

[20] Ryan Marcus and Olga Papaemmanouil. 2019. Plan-Structured Deep Neural Network Models for Query Performance Prediction. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1733–1746. doi:10.14778/3342263.3342646

[21] Zizhong Meng, Gao Cong, and Siqiang Luo. 2026. Are Learned DBMS Components Robust to Workload Drift? *Proceedings of the ACM on Management of Data* 4, 3 (2026), 1–24. doi:10.1145/3802014

[22] Toby J. Mitchell. 1974. An Algorithm for the Construction of D-Optimal Experimental Designs. *Technometrics* 16, 2 (1974), 203–210. doi:10.1080/00401706.1974.10489175

[23] Tatsuhiro Nakamori and Hideyuki Kawashima. 2026. Libra: One-Shot Parameter Sensitivity Estimation for Transfer Learning in Database Performance Prediction. *Proceedings of the VLDB Endowment* 19, 5 (2026), 945–957. doi:10.14778/3796195.3796207

[24] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[25] Maximilian Rieger and Thomas Neumann. 2025. T3: Accurate and Fast Performance Prediction for Relational Database Systems With Compiled Decision Trees. *Proceedings of the ACM on Management of Data* 3, 3, Article 227 (2025), 27 pages. doi:10.1145/3725364

[26] Ozan Sener and Silvio Savarese. 2018. Active Learning for Convolutional Neural Networks: A Core-Set Approach. In *Proceedings of the 6th International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1708.00489

[27] Tarique Siddiqui, Alekh Jindal, Shi Qiao, Hiren Patel, and Wangchao Le. 2020. Cost Models for Big Data Query Processing: Learning, Retrofitting, and Our Findings. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 99–113. doi:10.1145/3318464.3380584

[28] Ewout W. Steyerberg, Gerard J. J. M. Borsboom, Hans C. van Houwelingen, Marinus J. C. Eijkemans, and J. Dik F. Habbema. 2004. Validation and Updating of Predictive Logistic Regression Models: A Study on Sample Size and Shrinkage. *Statistics in Medicine* 23, 16 (2004), 2567–2586. doi:10.1002/sim.1844

[29] Ji Sun and Guoliang Li. 2019. An End-to-End Learning-based Cost Estimator. *Proceedings of the VLDB Endowment* 13, 3 (2019), 307–319. doi:10.14778/3368289.3368296

[30] Transaction Processing Performance Council. 2022. TPC Benchmark H (Decision Support) Standard Specification, Revision 3.0.1. https://www.tpc.org/tpch/ Data and queries from TPC-H tools 2.17.3 (gregrahn/tpch-kit, commit 852ad0a); later revisions changed only reporting and pricing rules.

[31] Transaction Processing Performance Council. 2024. TPC Benchmark DS Standard Specification, Version 4.0.0. https://www.tpc.org/tpcds/ Data and queries from TPC-DS tools 2.10.0 (gregrahn/tpcds-kit, commit 5a3a817).

[32] Pavel Valov, Jianmei Guo, and Krzysztof Czarnecki. 2020. Transferring Pareto Frontiers across Heterogeneous Hardware Environments. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering (ICPE)*. 12–23. doi:10.1145/3358960.3379127

[33] Pavel Valov, Jean-Christophe Petkovich, Jianmei Guo, Sebastian Fischmeister, and Krzysztof Czarnecki. 2017. Transferring Performance Prediction Models Across Different Hardware Platforms. In *Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE)*. 39–50. doi:10.1145/3030207.3030216

[34] Shivaram Venkataraman, Zongheng Yang, Michael Franklin, Benjamin Recht, and Ion Stoica. 2016. Ernest: Efficient Performance Prediction for Large-Scale Advanced Analytics. In *Proceedings of the 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 363–378. https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/venkataraman

[35] Yvonne Vergouwe, Daan Nieboer, Rianne Oostenbrink, Thomas P. A. Debray, Gordon D. Murray, Michael W. Kattan, Hendrik Koffijberg, Karel G. M. Moons, and Ewout W. Steyerberg. 2017. A Closed Testing Procedure to Select an Appropriate Method for Updating Prediction Models. *Statistics in Medicine* 36, 28 (2017), 4529–4539. doi:10.1002/sim.7179

[36] Wentao Wu, Yun Chi, Shenghuo Zhu, Junichi Tatemura, Hakan Hacıgümüş, and Jeffrey F. Naughton. 2013. Predicting Query Execution Time: Are Optimizer Cost Models Really Unusable?. In *Proceedings of the 29th IEEE International Conference on Data Engineering (ICDE)*. 1081–1092. doi:10.1109/ICDE.2013.6544899

[37] Ziniu Wu, Ryan Marcus, Zhengchun Liu, Parimarjan Negi, Vikram Nathan, Pascal Pfeil, Gaurav Saxena, Mohammad Rahman, Balakrishnan Narayanaswamy, and Tim Kraska. 2024. Stage: Query Execution Time Prediction in Amazon Redshift. In *Companion of the 2024 International Conference on Management of Data (SIGMOD-Companion)*. 280–294. doi:10.1145/3626246.3653391

[38] Henry P. Wynn. 1970. The Sequential Generation of D-Optimum Experimental Designs. *The Annals of Mathematical Statistics* 41, 5 (1970), 1655–1664. doi:10.1214/aoms/1177696809

[39] Yu Yan, Hongzhi Wang, Junfang Huang, Dake Zhong, Tao Yu, Kaixin Zhang, Man Yang, and Tianqing Wang. 2024. QCFE: An Efficient Feature Engineering for Query Cost Estimation. In *Proceedings of the 40th IEEE International Conference on Data Engineering (ICDE)*. 4302–4315. doi:10.1109/ICDE60146.2024.00328

[40] Jiani Yang, Sai Wu, Dongxiang Zhang, Jian Dai, Feifei Li, and Gang Chen. 2023. Rethinking Learned Cost Models: Why Start from Scratch? *Proceedings of the ACM on Management of Data* 1, 4, Article 255 (2023), 27 pages. doi:10.1145/3626769

[41] Yue Zhao, Gao Cong, Jiachen Shi, and Chunyan Miao. 2022. QueryFormer: A Tree Transformer Model for Query Plan Representation. *Proceedings of the VLDB Endowment* 15, 8 (2022), 1658–1670. doi:10.14778/3529337.3529349

[42] Zhanhao Zhao, Haotian Gao, Naili Xing, Lingze Zeng, Meihui Zhang, Gang Chen, Manuel Rigger, and Beng Chin Ooi. 2026. NeurBench: A Benchmark Suite for Learned Database Components with Drift Modeling. *Proceedings of the ACM on Management of Data* 4, 3 (2026), 1–27. doi:10.1145/3802091

[43] Qiang Zhu and Per-Åke Larson. 1994. A Query Sampling Method for Estimating Local Cost Parameters in a Multidatabase System. In *Proceedings of the 10th International Conference on Data Engineering (ICDE)*. 144–153. doi:10.1109/ICDE.1994.282996